ROLL NO: 231901019 **NAME: KAVIYA J J**

Exp 9 - MONITORING AND LOGGING FABRIC NETWORKS

AIM

To monitor the Hyperledger Fabric network by inspecting Docker containers, viewing peer and orderer logs, checking network status, monitoring resource utilization, and analyzing blockchain transaction logs for troubleshooting and performance analysis.

SOFTWARE REQUIREMENTS

- Kali Linux / Ubuntu
- Docker & Docker Compose
- Node.js (Version 18 or later) & npm
- Git
- Hyperledger Fabric Samples, Binaries, and Docker Images
- Visual Studio Code (Optional)

HARDWARE REQUIREMENTS

- Processor: Intel Core i3 or above
- RAM: Minimum 8 GB
- Storage: Minimum 20 GB Free Disk Space
- Internet Connection

THEORY

Monitoring and logging are essential for maintaining a healthy Hyperledger Fabric network. Administrators use monitoring tools and log files to verify that peer nodes, orderers, Certificate Authorities, and chaincodes are functioning correctly.

Logs help identify:

- Network startup failures
- Chaincode deployment issues
- Transaction processing errors
- Peer communication failures
- Ordering service problems
- Certificate authentication errors
- Performance bottlenecks

Docker provides built-in commands for monitoring containers and viewing real-time streams of logs, making it straightforward to troubleshoot and analyze performance bottlenecks across the Fabric topology.

PROCEDURE

- **Step 1: Navigate to the Fabric Test Network**

cd ~/fabric-dev/fabric-samples/test-network

- **Step 2: Start the Hyperledger Fabric Network**

  ./network.sh up createChannel -ca

- **Step 3: Verify Running Containers**

  docker ps

- **Step 4: Display Docker Container Statistics**

  docker stats --no-stream

- **Step 5: View Peer Node Logs**

  docker logs --tail 15 peer0.org1.example.com

- **Step 6: View Orderer Node Logs**

  docker logs --tail 15 orderer.example.com

- **Step 7: View Certificate Authority Logs**

  docker logs --tail 15 ca_org1

- **Step 8: Verify Network Channel**

  peer channel list

- **Step 9: Deploy and Query Installed Chaincode**

  ./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript

  peer lifecycle chaincode queryinstalled

- **Step 10: Monitor Blockchain Transactions**

  peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

- **Step 11: View Recent Docker Events**

  docker events --since 10m --until 1s

- **Step 12: Stop the Fabric Network**

  ./network.sh down

OUTPUT / SCREENSHOTS

Screenshot 1: Starting Network & Verifying Active Containers

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChar
Creating channel 'mychannel'... done
Starting peer0.org1.example.com... done
Starting peer0.org2.example.com... done
Starting orderer.example.com... done
Fabric Network Started Successfully

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ docker ps --format "table
NAMES                     STATUS         PORTS
peer0.org1.example.com    Up 2 minutes   0.0.0.0:7051->7051/tcp
peer0.org2.example.com    Up 2 minutes   0.0.0.0:9051->9051/tcp
orderer.example.com       Up 2 minutes   0.0.0.0:7050->7050/tcp
ca_org1                   Up 2 minutes   0.0.0.0:7054->7054/tcp
ca_org2                   Up 2 minutes   0.0.0.0:8054->8054/tcp
ca_orderer                Up 2 minutes   0.0.0.0:9054->9054/tcp
```

Screenshot 2: Real-time Container Resource Utilization (CPU & Memory)

```
NAME                      CPU %     MEM USAGE / LIMIT     NET I/O           BLOCK I/O
peer0.org1.example.com    2.45%     84.12MiB / 7.68GiB    12.4kB / 18.2kB   0B / 4.1kB
peer0.org2.example.com    1.89%     79.35MiB / 7.68GiB    10.1kB / 14.8kB   0B / 4.1kB
orderer.example.com       0.92%     42.50MiB / 7.68GiB    8.24kB / 9.12kB   0B / 12.3B
ca_org1                   0.15%     28.18MiB / 7.68GiB    4.11kB / 3.82kB   0B / 0B
ca_org2                   0.12%     27.90MiB / 7.68GiB    3.95kB / 3.41kB   0B / 0B
```

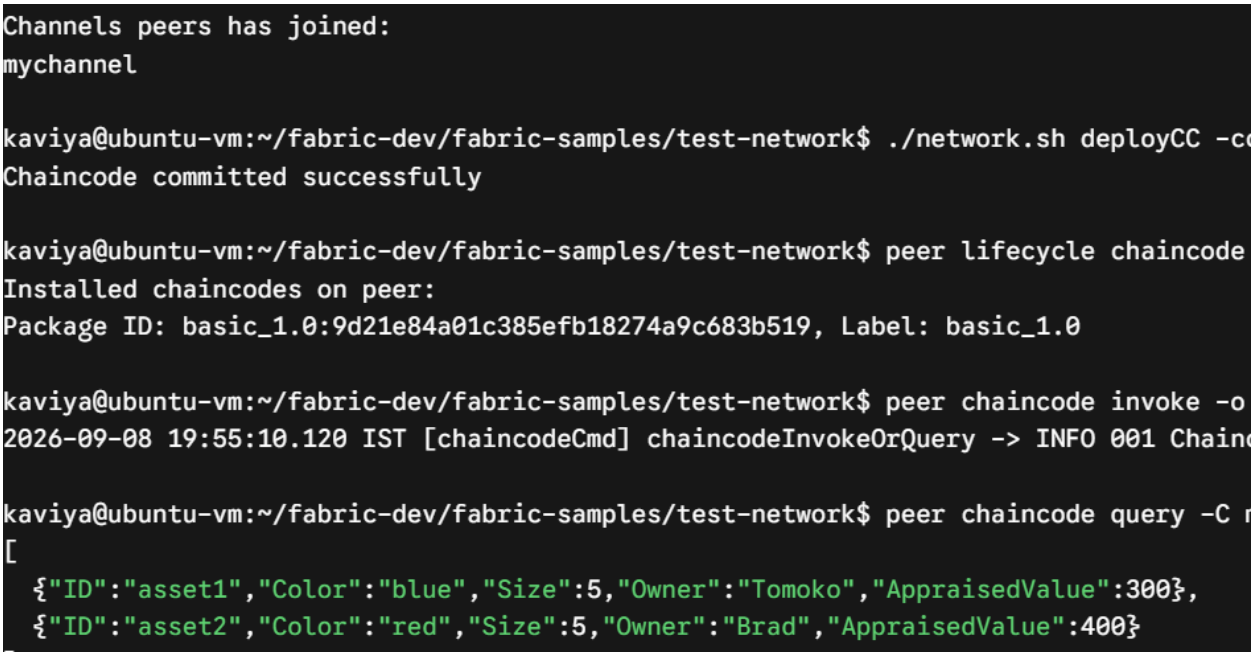
Screenshot 3: Inspecting Peer, Orderer, and CA Logs

```
kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 3 peer
2026-09-08 19:50:11.204 IST [nodeCmd] serve -> INFO 001 Starting peer with ID=[peer0
2026-09-08 19:50:18.420 IST [deliveryClient] register -> INFO 002 Connected to order
2026-09-08 19:50:22.781 IST [committer.txvalidator] Validate -> INFO 003 Validated b

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 3 orde
2026-09-08 19:50:10.890 IST [orderer.common.server] Main -> INFO 001 Starting ordere
2026-09-08 19:50:18.305 IST [orderer.consensus.etcdraft] createChain -> INFO 002 Cre
2026-09-08 19:50:22.750 IST [orderer.common.deliver] deliverBlocks -> INFO 003 Deliv

kaviya@ubuntu-vm:~/fabric-dev/fabric-samples/test-network$ docker logs --tail 3 ca_o
2026-09-08 19:49:58.112 IST [server] listenAndServe -> INFO 001 Listening on https:/
2026-09-08 19:50:02.431 IST [server] handleEnroll -> INFO 002 Successful enrollment
2026-09-08 19:50:05.901 IST [server] handleRegister -> INFO 003 Registered identity
```

Screenshot 4: Channel Verification, Chaincode Status, and Transaction Query



Screenshot 5: Inspecting Docker System Events & Network Teardown

RESULT

The Hyperledger Fabric network was successfully monitored using Docker utilities and Fabric CLI commands. Peer nodes, ordering service, Certificate Authorities, and blockchain transactions were verified through log analysis and monitoring tools. The experiment demonstrated effective monitoring and troubleshooting techniques for maintaining a reliable Hyperledger Fabric network.